

Final Project: Full-Stack Website Development

Courses: Web Technology in Business + Web Application Development

Program: IT Innovation in Business

Project Type: Individual

Submission Format: GitHub repository link + Technical Report + Presentation

Project Objective

Develop a secure, scalable, and functional full-stack web application that addresses a real-world business problem. The project must demonstrate proficiency in both front-end and back-end development, including modern best practices in software engineering, security, deployment, and testing.

Technology Stack

Frontend (choose one):

- React (preferred for single-page applications)
- Django Templates (preferred for tightly coupled or server-rendered apps)

Backend (choose one):

- Django
- Django REST Framework (DRF)

Functional & Technical Requirements

Authentication & Authorization

- User registration, login, logout
- Role-based access control (e.g., admin, user)
- Secure password handling (hashing, password reset functionality)
- Auth middleware or decorators (e.g., `@login_required`, DRF permission)



Content Management

- Admin interface for managing core app content (via Django Admin or custom CMS)
- CRUD operations for business-relevant models (e.g., products, posts, clients)



Security

- Protection against XSS, CSRF, and SQL injection
- Environment variables hidden using `.env` and `.gitignore`



Testing

- Unit tests for backend logic (models, views, serializers, etc.)
- Integration or end-to-end test flows using Django test client or tools like Selenium/Cypress
- Front-end testing



Dockerization

- Dockerize both frontend and backend
- Use `docker-compose` to orchestrate services (frontend, backend, database, Redis)
- Use a production-ready web server/proxy like **Nginx**
- Separate `.env` configurations for dev and prod environments



Logging

- Implement structured logging in backend
- Log to file or stdout
- Use log levels appropriately (**INFO**, **DEBUG**, **WARNING**, **ERROR**)



Caching

- Use Redis (or similar) to cache expensive queries or frequently accessed data
 - Example: user sessions, product lists, or homepage data
- Document caching strategy in `README.md`



Documentation & Code Quality

- Fill out a clear and structured `README.md` including:
 - Project description
 - Setup for development and production
 - Testing instructions
 - Docker instructions
 - Caching/logging notes
- Use linters (e.g., `flake8`, `black`, `eslint`) for consistent code style

✓ Evaluation Criteria (200 points)

Category	Max Points
Functional Requirements Met	15
Authentication & Authorization	10
Secure Coding Practices (HTTPS, env, CSRF)	10
Code Quality & Folder Structure	5
Unit & Integration/E2E Testing	5
Dockerization (multi-service, nginx, .env)	10
Logging Implementation	5
Caching Implementation	5
README & Documentation	10
Report & Presentation	20
Bonus: Live Deployment	5
Project Defense	100

📁 Deliverables

GitHub Repository

- Clean and modular folder structure
- Fully functional Docker setup with Dockerfiles and `docker-compose.yml`
- `.env.example` and `.gitignore` configured properly
- Full test suite and instructions to run tests
- `README.md` with setup, usage, and development instructions

Report & Presentation

Report Sections:

1. **Introduction:** Project goals, business use case, and problem statement
2. **System Design:** Functional architecture, block diagram, use cases, key features
3. **Implementation:** Technology choices, key implementation notes
4. **Testing & Deployment:** Summary of tests, Docker setup, deployment (if applicable)
5. **Conclusion & Literature:** Lessons learned, future work, references

Presentation (8–10 minutes):

- Problem statement
- Key features
- Live demo or screenshots
- Architecture and tools used
- Summary and reflections



Timeline

- **Final Submission Deadline: 05/06/2025**



Important Note:

- Submission of the **technical report and presentation is mandatory**. These are not optional deliverables — they are essential components of the final project.
- While the report and presentation are assessed based on their quality, **failure to submit them will result in the project being considered incomplete and therefore not accepted**.
- Your project must also be **fully runnable** — I should be able to **clone your repository and run the application locally** using the instructions provided in your [README.md](#).
- Finally, you will be required to **defend your project during a final presentation session** by answering questions related to your design decisions, implementation, security, testing, and architecture.